INFORME TÉCNICO Y FUNCIONAL

EcoHuella

Aplicación web para estimar la huella de carbono personal: qué hace, cómo está construida, cómo calcula y cómo se despliega.

React 19 · Vite 8 · Tailwind CSS 4 · Node.js + Express · MongoDB · JWT · PWA

Documento
Informe de la aplicación completa (frontend, backend, datos, seguridad y despliegue)

Fecha de elaboración
14 de octubre de 2026

Código analizado
Repositorio *eco-huella*, rama *main* (commit cbee746) más los cambios locales y las correcciones de esta revisión (sin commitear)

Método
Lectura completa del código, pruebas del backend contra una base temporal, verificación en navegador y ejecución de la lógica de cálculo

Índice de contenidos

1. Resumen ejecutivo 4
- 1.1 Qué hace la aplicación, en una frase por pantalla 4
- 1.2 Estado general 5

2. Contexto: qué problema aborda 5
- 2.1 La huella de carbono, explicada sin tecnicismos 5
- 2.2 Valores de referencia que usa la aplicación 5
- 2.3 Alcance de la medición 6

3. Tecnologías utilizadas 6
- 3.1 Frontend (lo que se ejecuta en el navegador) 6
- 3.2 Backend (lo que se ejecuta en el servidor) 6
- 3.3 Plataformas de despliegue 7

4. Arquitectura general 7
- 4.1 Desarrollo local frente a producción 8
- 4.2 Ciclo de vida de una petición autenticada 8

5. Estructura del proyecto 8
- 5.1 Patrón de organización del backend 9

6. El frontend en detalle 10
- 6.1 Arranque de la aplicación 10
- 6.2 Rutas y control de acceso 10
- 6.3 Sesión y estado global (AuthContext) 11
- 6.4 Capa de servicios (cliente HTTP) 11
- 6.5 Las pantallas, una por una 12
- 6.6 Diseño visual y componentes de interfaz 18
- 6.7 Datos estáticos que alimentan el cuestionario 19
- 6.8 Aplicación instalable (PWA) y funcionamiento sin conexión 20

7. Flujo completo del usuario 20
- 7.1 Flujos secundarios 21

8. El motor de cálculo 22
- 8.1 Fórmulas por categoría 22
- 8.2 Métricas derivadas 23
- 8.3 Ejemplo resuelto paso a paso 23
- 8.4 Resultados con tres perfiles distintos 24
- 8.5 Qué se guarda en el servidor 25

9. El backend en detalle 25
- 9.1 Servidor Express (server.js) 25
- 9.2 Modelos de datos 25
- 9.3 Catálogo de endpoints 26
- 9.4 Autenticación con JWT 27
- 9.5 Recuperación de contraseña 28
- 9.6 Roles y permisos 28
- 9.7 Variables de entorno 29
- 9.8 Script de datos iniciales (seed.js) 29

10. Base de datos 29
10.1 Reglas de integridad 30
10.2 Ejemplo de documento de resultado 30

11. Seguridad y privacidad 30
11.1 Medidas implementadas 30
11.2 Datos personales que trata la aplicación 31

12. Despliegue y operación 32
12.1 Cómo se publica 32
12.2 Scripts disponibles 32
12.3 Consideraciones operativas 32

13. Calidad del código y estado del proyecto 33
13.1 Resumen 33
13.2 Cómo evolucionó (historial de commits) 33

14. Correcciones aplicadas y observaciones pendientes 35
14.1 Correcciones aplicadas 35
14.2 Cómo se verificó 36
14.3 Efecto de las correcciones en los resultados 37
14.4 Observaciones pendientes 38
14.5 Plan de acción sugerido 39

15. Cómo ejecutarla en una computadora 39
15.1 Requisitos 39
15.2 Pasos 39
15.3 Verificación y problemas frecuentes 40

16. Glosario 40

1. Resumen ejecutivo

EcoHuella es una aplicación web que permite a una persona **estimar su huella de carbono anual** respondiendo encuestas breves sobre sus hábitos. Con esas respuestas calcula cuántas toneladas de CO_2 equivalente (tCO_2e) emite por año, la compara con una media de referencia, estima cuántos "planetas Tierra" harían falta si todos vivieran igual y sugiere acciones concretas para reducir el impacto.

> **Este informe describe la aplicación ya corregida**
> Durante la revisión se detectaron errores en el cálculo, la navegación y la seguridad. Los que eran defectos claros se corrigieron y verificaron (sección 14), así que las capturas, fórmulas y resultados de este documento reflejan el código actual.

3
recorridos de encuesta (ambiental, agua, desayuno)

21
preguntas en total (14 + 3 + 4)

26
endpoints de negocio de la API REST (más 4 URL de salud)

4
colecciones en MongoDB (usuarios, resultados, preguntas, parámetros)

25
componentes React (archivos .jsx), 3 de ellos vacíos

48
commits entre el 26/04/2026 y el 14/05/2026

1.1 Qué hace la aplicación, en una frase por pantalla

Pantalla	Qué hace
Registro / Inicio de sesión	Crea cuentas con un perfil completo (edad, ubicación, nivel educativo, hogar) y autentica con correo *o* nombre de usuario. Permite recuperar la contraseña con un código de 6 dígitos enviado por correo.
Términos y condiciones	Obliga a aceptar las condiciones de uso antes de entrar a la aplicación.
Inicio	Presenta la propuesta, explica qué es el CO_2e y responde preguntas frecuentes.
Selector de encuestas	Ofrece tres recorridos; cada uno puede completarse *una sola vez* por usuario.
Cuestionario	Presenta una pregunta por pantalla con barra de progreso y guarda el resultado al finalizar.
Dashboard	Muestra huella total, comparación con la media, desglose por categoría, planetas necesarios, historial y acciones sugeridas.
Perfil	Permite editar datos personales, contraseña y foto (con zoom y desplazamiento).
Panel de administración	Solo para administradores: gestionar usuarios y roles, preguntas y parámetros de emisión.

1.2 Estado general

Lo que está bien resuelto

- Arquitectura clara en dos capas (SPA React + API REST) con separación en carpetas por responsabilidad.
- Autenticación con JWT, contraseñas con hash (bcrypt) y roles *user*/*admin* aplicados en el servidor.
- Recuperación de contraseña con código hasheado y vencimiento de 15 minutos.
- Regla de "una encuesta por tipo" verificada en cliente y en servidor (HTTP 409).
- Instalable como PWA, diseño responsivo y guía de despliegue gratuito documentada.

Corregido y pendiente (detalle en la sección 14)

- **Corregido:** el cálculo del transporte en auto y del desperdicio (un perfil típico pasó de 10,7 t a 3,7 t), la redirección al recargar, los mensajes con codificación corrupta y varios puntos de seguridad.
- **Pendiente:** el panel de preguntas y parámetros no está conectado al cuestionario real y el consentimiento de términos se guarda solo en el navegador.

> **Cómo leer este informe**
> Las secciones 2 a 5 explican el contexto, la tecnología y la arquitectura. Las secciones 6 a 10 describen el funcionamiento por dentro (frontend, cálculo, backend y datos). Las secciones 11 a 14 evalúan seguridad, despliegue y calidad, y documentan las correcciones aplicadas y las mejoras pendientes. La sección 15 explica cómo ejecutarla y la 16 es un glosario para quien no sea del área técnica.

2. Contexto: qué problema aborda

2.1 La huella de carbono, explicada sin tecnicismos

Casi todo lo que hacemos libera gases de efecto invernadero: usar electricidad, calefaccionar, movernos, comer, comprar y desechar. Esos gases se miden de forma conjunta en **CO_2 equivalente (CO_2e)**: una unidad común que traduce el efecto de cada gas al de una cantidad de dióxido de carbono. La **huella de carbono personal** es la suma anual de esas emisiones, y se expresa en **toneladas de CO_2e por año** (t CO_2e/año).

2.2 Valores de referencia que usa la aplicación

Referencia	Valor	Dónde aparece y para qué se usa
Límite sostenible	1,5 t/año	Se presenta en la pantalla de inicio como "objetivo anual por persona". En el cálculo se usa como divisor para estimar los *planetas Tierra* necesarios (total ÷ 1,5).
Media nacional	4,7 t/año	Constante del código (`quizResults.js`) usada para calcular la diferencia porcentual del usuario respecto a la media. La aplicación no cita la fuente del dato.
Media global	> 4 t/año	Mencionada solo como texto informativo en la pantalla de inicio.

2.3 Alcance de la medición

La aplicación mide cuatro categorías principales —**hogar, transporte, alimentación y residuos**— y dos recorridos complementarios: **consumo de agua** (lavarropas, riego y lavado de auto) y **desayuno** (bebida, comida, origen y desperdicio). Es una herramienta **educativa y orientativa**: los propios Términos y Condiciones (sección 6) aclaran que los resultados son estimaciones y no una auditoría técnica.

> **Contexto institucional**
> El perfil de usuario incluye el campo **"Nivel UCCuyo"** (jardín, primario, secundaria, universidad), lo que sugiere un uso en el ámbito educativo de esa institución. El código no documenta explícitamente este contexto; se menciona aquí porque condiciona qué datos personales se recolectan (ver sección 11).

3. Tecnologías utilizadas

La aplicación está escrita íntegramente en **JavaScript** (módulos ES) tanto en el navegador como en el servidor. Las versiones corresponden a las declaradas en los `package.json`.

3.1 Frontend (lo que se ejecuta en el navegador)

Tecnología	Versión	Para qué se usa en el proyecto
React + React DOM	19.2	Biblioteca para construir la interfaz con componentes reutilizables y estado.
React Router DOM	6.30	Navegación entre pantallas sin recargar la página (SPA) y rutas protegidas.
Axios	1.16	Cliente HTTP; un interceptor agrega el token JWT a cada petición.
Vite	8.0	Servidor de desarrollo con recarga instantánea y empaquetador para producción. Hace de proxy de `/api` hacia el backend en desarrollo.
Tailwind CSS	4.2	Utilidades de estilo y "tokens" de diseño propios (colores, tipografías, radios). Se complementa con ~4.400 líneas de CSS propio en `index.css`.
ESLint	10	Análisis estático del código (con reglas de React Hooks y React Refresh).
Inter, Plus Jakarta Sans, Material Symbols	—	Tipografías e íconos cargados desde Google Fonts.

3.2 Backend (lo que se ejecuta en el servidor)

Tecnología	Versión	Para qué se usa en el proyecto
Node.js	≥ 20	Entorno de ejecución de JavaScript en el servidor.
Express	4.18	Framework web: define rutas, middlewares y respuestas JSON.
MongoDB + **Mongoose**	Mongoose 7.6	Base de datos documental y su ODM, que aporta esquemas y validaciones.
jsonwebtoken	9.0	Emisión y verificación de tokens JWT (sesión sin estado).

Tecnología	Versión	Para qué se usa en el proyecto
bcryptjs	2.4	Hash de contraseñas con sal (10 rondas).
Nodemailer y API de **Resend**	8.0	Envío del correo con el código de recuperación (Resend por HTTP; Gmail/SMTP como alternativa).
cors, dotenv	2.8 / 16	Permitir peticiones desde otro origen y leer variables de entorno.
express-validator	7.0	Instalado pero *no utilizado* en el código actual.

3.3 Plataformas de despliegue

Servicio	Rol
Vercel	Hospeda el frontend compilado (archivos estáticos). Un *rewrite* envía cualquier ruta a `index.html` para que funcione el enrutado del lado del cliente.
Render	Ejecuta el backend Node/Express (plan gratuito; puede "dormirse" tras un período sin uso).
MongoDB Atlas	Base de datos en la nube (plan gratuito).
Resend / Gmail SMTP	Envío de correos transaccionales.

4. Arquitectura general

EcoHuella sigue una arquitectura clásica **cliente–servidor de tres capas**: una aplicación de página única (SPA) en React, una API REST en Express y una base de datos MongoDB. El navegador nunca habla con la base de datos: todo pasa por la API, que valida la identidad y los permisos.

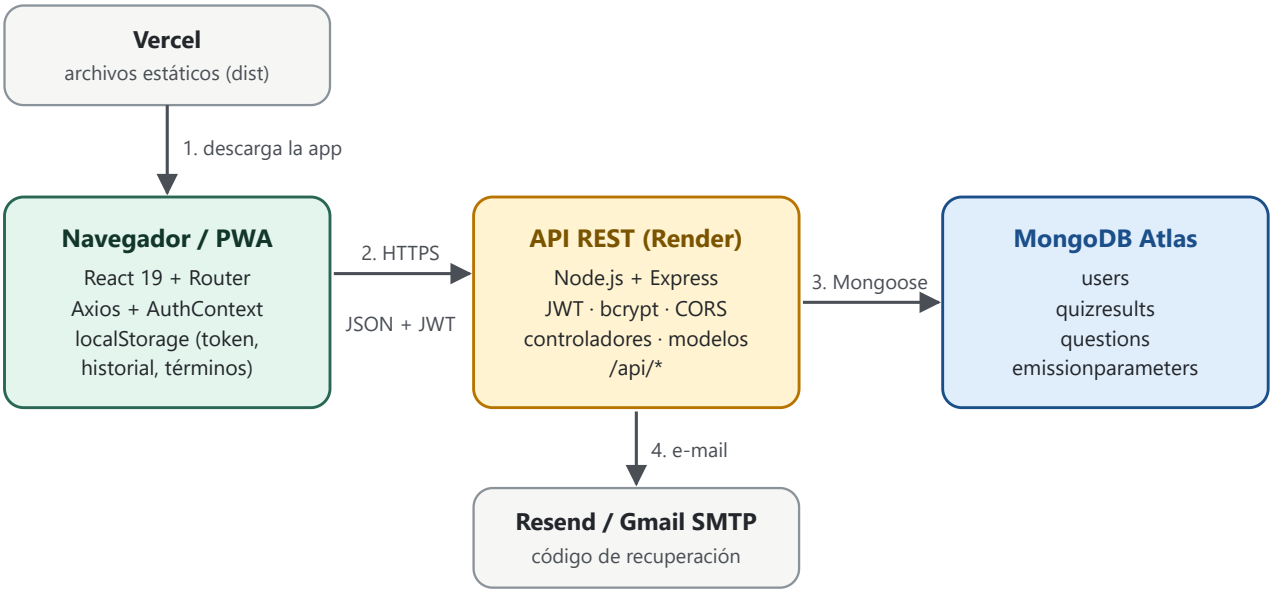


Figura 1. Componentes del sistema en producción y el sentido de las comunicaciones.

4.1 Desarrollo local frente a producción

Aspecto	Desarrollo local	Producción
Frontend	Vite en `http://localhost:8080` con recarga en caliente.	Build estático (`npm run build` → `dist/`) servido por Vercel.
Backend	Express en `http://localhost:5000` (`node --watch`).	Express en Render (`npm start`, carpeta raíz `backend`).
Conexión front→API	El frontend llama a `/api` y el *proxy* de Vite lo reenvía a `localhost:5000` (evita problemas de CORS).	Usa `VITE_API_BASE_URL` si está definida; si no, una URL de Render fijada en `api.js`.
Base de datos	MongoDB local (`mongodb://localhost:27017/eco-huella`).	MongoDB Atlas mediante `MONGODB_URI`.
Service worker	No se registra.	Se registra (solo si `import.meta.env.PROD`).

4.2 Ciclo de vida de una petición autenticada

1. Un componente llama a una función de `services/api.js` (por ejemplo `quizResultsAPI.getMyResults()`).
2. El **interceptor de Axios** lee `token` de `localStorage` y agrega el encabezado `Authorization: Bearer <token>`.
3. En el servidor, Express aplica `cors()` y `express.json()`, y el middleware `requireDatabase` responde 503 si MongoDB no está conectado.
4. El middleware `auth` verifica la firma del JWT y deja `userId` y `role` en la petición; para rutas de admin, `isAdmin` exige `role === "admin"`.
5. El controlador consulta o modifica la base mediante un modelo Mongoose y responde JSON.
6. Ante un error no controlado, un manejador global responde 500 sin exponer la traza.

5. Estructura del proyecto

El repositorio contiene en una misma carpeta el frontend (en la raíz y en `src/`) y el backend (en `backend/`), cada uno con su propio `package.json`.

```
eco-huella/
├── index.html              # documento base: fuentes, manifest, metadatos PWA
├── vite.config.js          # puerto 8080 y proxy /api → :5000
├── tailwind.config.js      # tokens de diseño (colores, radios, tipografías)
├── vercel.json             # rewrite SPA para Vercel
├── public/                 # sw.js, manifest.webmanifest, íconos
├── src/
│   ├── main.jsx            # arranque de React + registro del service worker
│   ├── App.jsx             # definición de rutas
│   ├── index.css           # estilos globales (≈4.400 líneas)
│   ├── context/AuthContext.jsx     # sesión, perfil y aceptación de términos
│   ├── services/api.js             # cliente Axios y funciones por recurso
│   ├── utils/quizResults.js        # motor de cálculo de la huella
│   ├── utils/quizStorage.js        # historial local de encuestas
│   ├── data/questions.json         # preguntas y factores de emisión
│   ├── data/surveyTracks.js        # los 3 recorridos de encuesta
│   └── components/
│       ├── auth/       LoginPage, SignupPage
│       ├── features/   LandingPage, SurveySelector, QuizController,
│       │               QuestionCard, ResultsDashboard
│       ├── admin/      AdminPanel, UserManagement,
│       │               QuestionManagement, ParameterManagement
│       ├── profile/    ProfileModal
│       ├── legal/      TermsAndConditionsPage
│       ├── layout/     Layout (cabecera, navegación, pie)
│       └── ui/         Button, Card
└── backend/
    ├── server.js           # Express, middlewares, rutas, errores
    ├── seed.js             # carga datos iniciales (destructivo)
    ├── config/database.js  # conexión a MongoDB
    ├── models/             # User, QuizResult, Question, EmissionParameter
    ├── controllers/        # lógica de cada recurso
    ├── routes/             # mapeo URL → controlador
    ├── middleware/auth.js  # auth e isAdmin
    └── utils/emailService.js
```

5.1 Patrón de organización del backend

El backend sigue el patrón **Ruta** → **Controlador** → **Modelo**: la ruta decide qué middleware aplica y a qué función delega; el controlador contiene la lógica y devuelve la respuesta; el modelo define la forma y las reglas de los datos. Esta separación facilita agregar un recurso nuevo sin tocar los existentes.

6. El frontend en detalle

El frontend es una **aplicación de página única (SPA)**: el navegador descarga una sola vez el HTML y el JavaScript, y desde ese momento React cambia el contenido de la pantalla sin recargar la página. Todo el código fuente está en `src/`.

6.1 Arranque de la aplicación

El punto de entrada es `src/main.jsx`. Realiza tres tareas:

1. **Monta React** en el elemento `#root` dentro de `StrictMode` (que detecta efectos secundarios no deseados en desarrollo).
2. **Protege los íconos de la traducción automática.** Un `MutationObserver` marca cada ícono de Material Symbols con `translate="no"`. Sin esto, el traductor del navegador convierte el nombre del ícono (texto) en palabras traducidas y rompe la interfaz.
3. **Registra el service worker** `/sw.js` solo en producción. Si detecta una versión nueva ya activada, recarga la página una única vez por sesión.

6.2 Rutas y control de acceso

`App.jsx` declara las rutas con React Router. Las pantallas internas se envuelven en `ProtectedRoute` (control de acceso) y `Layout` (cabecera y pie comunes).

Ruta	Componente	Acceso	Observaciones
`/login`	LoginPage	**Pública**	Incluye el modal de recuperación de contraseña.
`/signup`	SignupPage	**Pública**	Registro con perfil completo.
`/terminos-y-condiciones`	TermsAndConditionsPage	**Con sesión**	Sin sesión redirige a `/login`; si ya aceptó, a `/dashboard`.
`/`	LandingPage	**Protegida**	Pantalla de inicio informativa.
`/encuestas`	SurveySelector	**Protegida**	Elección del recorrido.
`/quiz/:trackId`	QuizController	**Protegida**	`trackId` ∈ ambiental, agua, desayuno. Un valor desconocido cae en "ambiental".
`/quiz`	—	—	Redirige a `/encuestas`.
`/dashboard`	ResultsDashboard	**Protegida**	Resultados e historial.
`/admin`	AdminPanel	**Solo admin**	Un usuario sin rol admin es enviado a `/dashboard`.
`*`	—	—	Cualquier otra URL redirige a `/`.

`ProtectedRoute` aplica, en este orden: sin sesión → `/login`; sesión en verificación → indicador de carga; términos pendientes o no aceptados → `/terminos-y-condiciones`; ruta de admin sin rol → `/dashboard`. Esperar a que se confirme la sesión (estado `checkingAuth`) es lo que permite recargar cualquier pantalla interna sin perder la ruta.

> **Importante: el control del frontend es de comodidad, no de seguridad**
>
> Ocultar una pantalla en el navegador no impide que alguien llame directamente a la API. La seguridad real está en el backend (middlewares `auth` e `isAdmin`, sección 9.6). El frontend solo evita mostrar opciones que el usuario no puede usar.

6.3 Sesión y estado global (AuthContext)

`context/AuthContext.jsx` es la pieza central. Expone mediante el hook `useAuth()` el usuario actual, el token, los indicadores `isAuthenticated`, `isAdmin` y `checkingAuth` (sesión en verificación), el estado de aceptación de términos y las acciones `signup`, `login`, `logout`, `updateProfile`, `acceptTerms` y `rejectTerms`.

Al cargar la aplicación, si existe un token guardado, el contexto pide `GET /auth/profile` para reconstruir el usuario; si el servidor rechaza el token, la sesión se descarta. Mientras dura esa consulta, `checkingAuth` es verdadero.

Datos que la aplicación guarda en el navegador

Clave	Contenido
`localStorage.token`	JWT de la sesión.
`ecohuella_terms_accepted_v1`	Lista de identificadores de usuario que aceptaron los términos *en este navegador*.
`ecohuella_terms_pending_user_v1`	Id del usuario recién registrado que aún debe aceptar los términos.
`ecohuella_quiz_answers_<id>`	Últimas respuestas del usuario.
`ecohuella_quiz_history_<id>`	Historial local de intentos (máximo 20).
`quizAnswers`	Clave heredada de versiones anteriores; se elimina al registrarse o terminar una encuesta.
`sessionStorage.ecohuella_sw_refreshed`	Evita recargas repetidas al actualizarse el service worker.

6.4 Capa de servicios (cliente HTTP)

`services/api.js` crea una instancia de Axios y agrupa las llamadas por recurso. La URL base se resuelve así: `VITE_API_BASE_URL` si existe; en producción, una URL de Render definida en el archivo; en desarrollo, `/api` (que Vite redirige al backend). Un interceptor agrega el token a cada petición.

Grupo	Funciones	Funciones disponibles
`authAPI`	6	signup, login, forgotPassword, resetPassword, getProfile, updateProfile
`questionsAPI`	5	getAll, getById, create, update, delete
`parametersAPI`	4	getAll, getByCategory, update, create
`quizResultsAPI`	6	getMyResults, getById, save, update, delete, getAllResults
`usersAPI`	5	getAll, getById, update, delete, changeRole

6.5 Las pantallas, una por una

6.5.1 Inicio de sesión y recuperación de contraseña

Acepta **correo o nombre de usuario** más la contraseña. Incluye botón para mostrar/ocultar la contraseña y campos "señuelo" invisibles que evitan que el navegador autocomplete datos. El checkbox "Mantenerme conectado" existe visualmente pero no cambia el comportamiento: el token siempre queda guardado.

La recuperación se hace en un modal de **tres pasos**: (1) ingresar el correo registrado, (2) escribir el código de 6 dígitos recibido y la nueva contraseña con su confirmación, (3) confirmación de éxito.

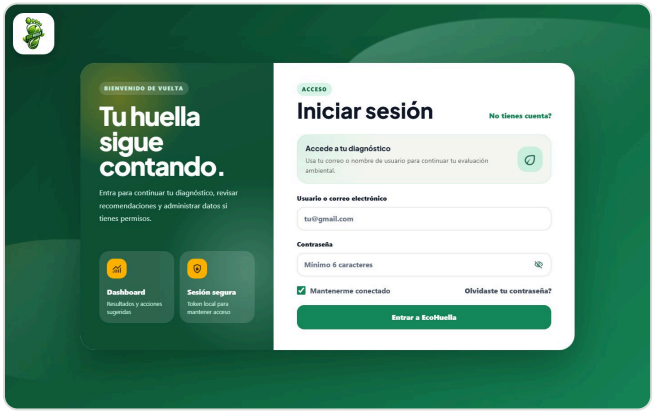


Figura 2. Inicio de sesión.

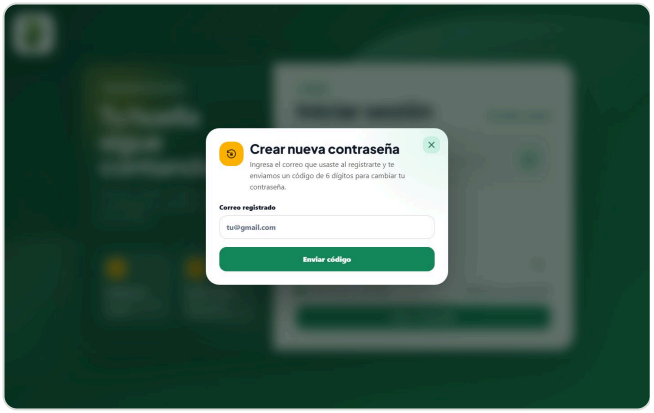


Figura 3. Modal de recuperación de contraseña (paso 1).

6.5.2 Registro

Solicita nombre, usuario, correo, fecha de nacimiento (la edad se calcula sola), sexo, ciudad, provincia, país, nivel UCCuyo, ocupación, personas en el hogar, la meta de sostenibilidad y la contraseña con confirmación. **Validaciones del cliente:** contraseñas iguales y de al menos 6 caracteres; usuario de 3 a 24 caracteres con letras, números o guion bajo; fecha de nacimiento válida (0–120 años).

Al registrarse, el servidor devuelve token y usuario; la aplicación marca los términos como *pendientes* y envía a `/terminos-y-condiciones`.

6.5.3 Términos y condiciones

Muestra 13 apartados (identificación del servicio, aceptación, registro y veracidad, uso permitido, datos personales y privacidad, resultados y limitaciones, propiedad intelectual, responsabilidades, disponibilidad, administración, conservación de datos, cambios y contacto). **Aceptar** registra la aceptación en el navegador y entra al dashboard; **No acepto** cierra la sesión.

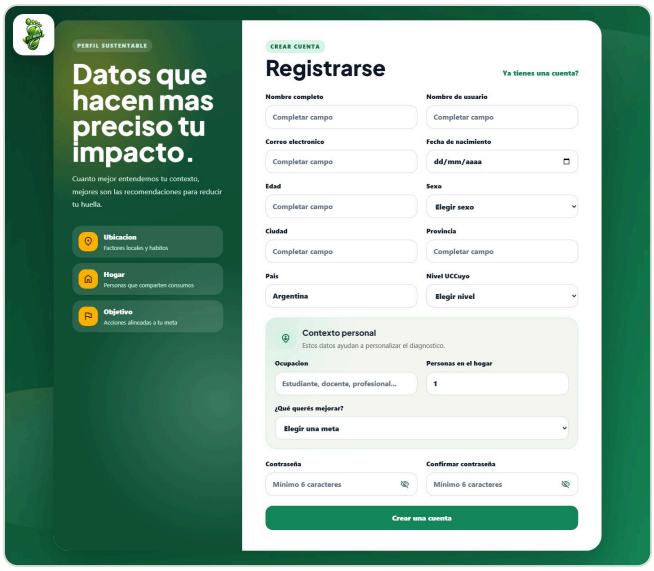


Figura 4. Formulario de registro.



Figura 5. Términos y condiciones, previos al acceso.

6.5.4 Inicio (landing)



Figura 6. Pantalla de inicio (captura de página completa).

Página informativa y estática (no consulta datos al servidor), organizada en cuatro bloques:

1. **Portada:** título y botón *Comenzar Cuestionario*, que lleva a `/encuestas`.
2. **Tu viaje hacia la sostenibilidad:** tarjetas sobre el límite sostenible de 1,5 t, qué es el CO_2e y el enfoque "menos culpa, más acción".
3. **Un diagnóstico de 3 minutos:** cinco tarjetas de método (hogar, agua, movilidad, alimentación y residuos).
4. **Preguntas frecuentes:** tres preguntas desplegables sobre el uso de los datos, el origen de los cálculos y qué hacer ante una huella alta.

El botón principal es el único elemento interactivo relevante; el resto orienta a la persona antes de comenzar.

6.5.5 Selector de encuestas y cuestionario

El selector muestra los tres recorridos definidos en `surveyTracks.js`. Un recorrido ya completado aparece deshabilitado con un tilde. Para saberlo, combina las respuestas del servidor (`GET /results`) con el historial local.

El cuestionario muestra **una pregunta por pantalla** con módulo, barra de progreso y botones *Atrás*/*Siguiente* (deshabilitado hasta responder). Soporta tres tipos de entrada: **select** (tarjetas con ícono), **number** (campo numérico) y **range** (deslizador, p. ej. 0–500 km en pasos de 10).

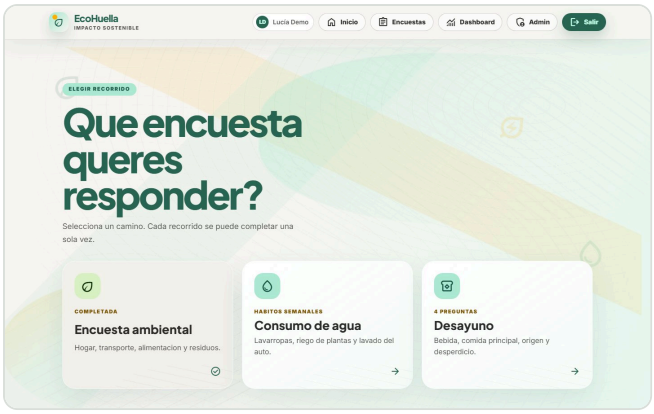


Figura 7. Selector de recorridos (la encuesta ambiental figura como completada).

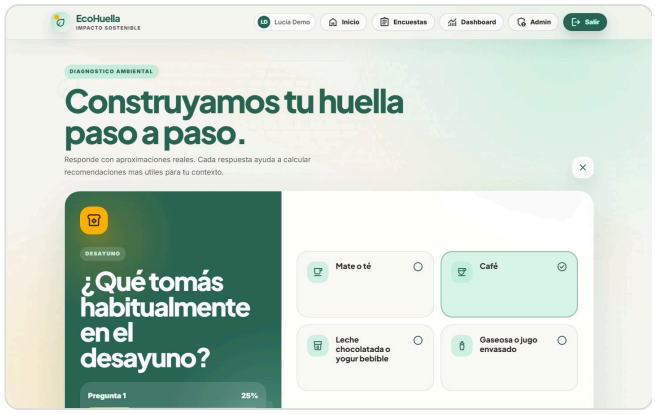


Figura 8. Cuestionario del recorrido "Desayuno" con una opción seleccionada.

> **Qué ocurre al responder la última pregunta**
>
> 1. Se guarda el intento en el historial local del navegador.
> 2. Se calcula el resultado y se envía al servidor (`POST /results`) junto con las respuestas.
> 3. Si el envío falla, el error solo se registra en consola y el usuario igualmente pasa al dashboard con sus datos locales.
> 4. Se navega a `/dashboard`.

6.5.6 Dashboard de resultados

Es la pantalla más compleja (`ResultsDashboard.jsx`, 440 líneas). Muestra:

- **Huella total** en t CO_2e y la diferencia porcentual contra la media nacional (verde si está por debajo, rojo si por encima). Los recorridos de agua y desayuno miden solo una parte de la huella, por eso no se comparan con la media: se muestra una explicación.
- **Consumo de agua estimado** en m^3/año, tomado de la encuesta de agua más reciente del historial (muestra "—" si aún no se completó).
- **Desglose por categoría**: barras proporcionales para hogar, transporte, alimentación y residuos.
- **Planetas Tierra necesarios**: un ícono por planeta entero más uno parcial (solo para la encuesta ambiental).
- **Historial de encuestas**: tarjetas clicables para ver un resultado anterior.
- **Acciones sugeridas**: dos recomendaciones para la categoría de mayor impacto.

Cómo decide qué mostrar: reúne los intentos del servidor y los locales, elimina duplicados comparando sus respuestas, toma el más reciente (o el elegido) y recalcula el resultado desde las respuestas; si el intento no tiene respuestas utilizables, usa los valores ya guardados. Si no hay datos, muestra un estado vacío que invita a iniciar la encuesta.



Figura 9. Dashboard con datos de demostración (usuario ficticio).

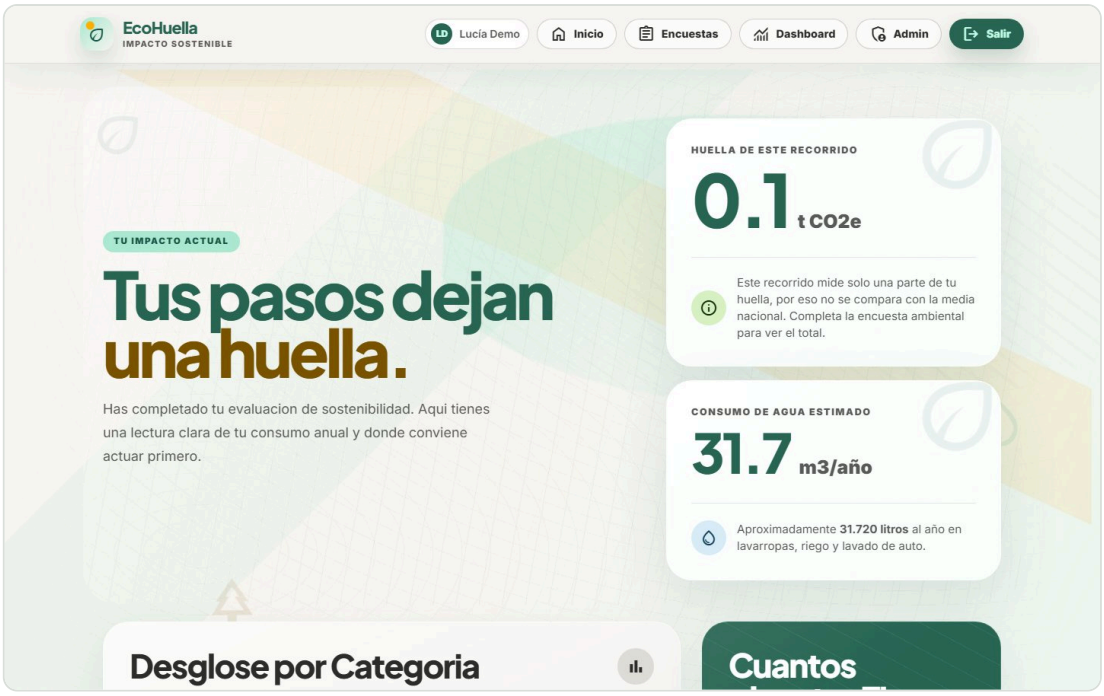


Figura 10. Resultado de un recorrido parcial (desayuno): no se compara con la media nacional y se explica el motivo.

6.5.7 Perfil de usuario

Un modal abierto desde el chip del usuario en la cabecera. Permite editar todos los datos personales y la contraseña (opcional). La **foto de perfil** se lee en el navegador como imagen en base64 (máximo 8 MB), con controles de zoom (1–3×) y desplazamiento horizontal/vertical (±50 %); esos valores se guardan junto al usuario.

6.5.8 Panel de administración

Disponible solo para el rol *admin*, con tres pestañas y un botón para refrescar.

Pestaña	Qué permite
Usuarios	Listar todos los perfiles, editar sus datos (incluida una nueva contraseña), cambiar el rol desde un selector y eliminar con confirmación. La interfaz no incluye alta de usuarios: el formulario solo edita existentes.
Preguntas	Crear, editar y eliminar preguntas con id, módulo, tipo, orden y texto. No permite cargar opciones ni factores.
Parámetros	Ver los cuatro grupos de factores de emisión y editar sus valores numéricos.

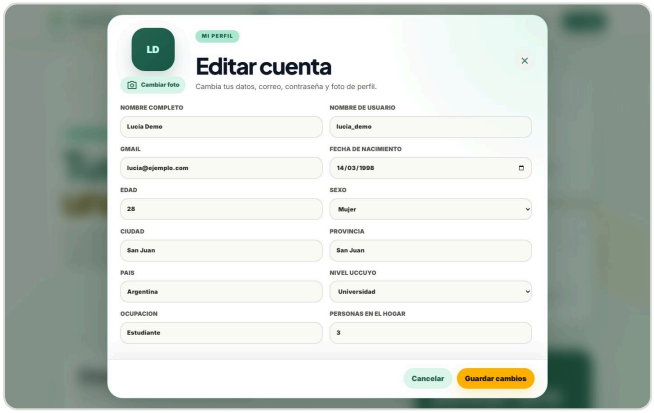


Figura 11. Modal de edición de perfil.

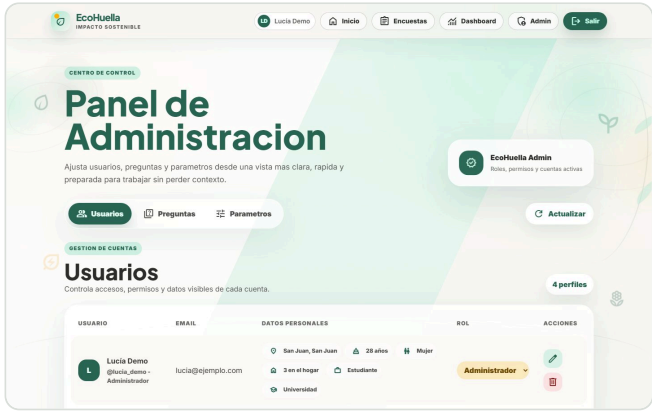


Figura 12. Panel de administración: pestaña Usuarios.

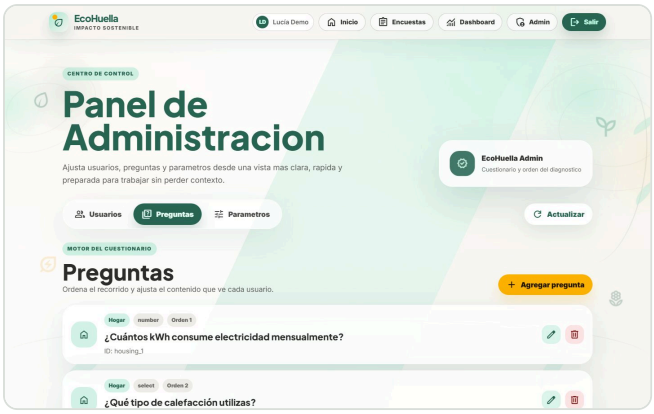


Figura 13. Pestaña Preguntas.

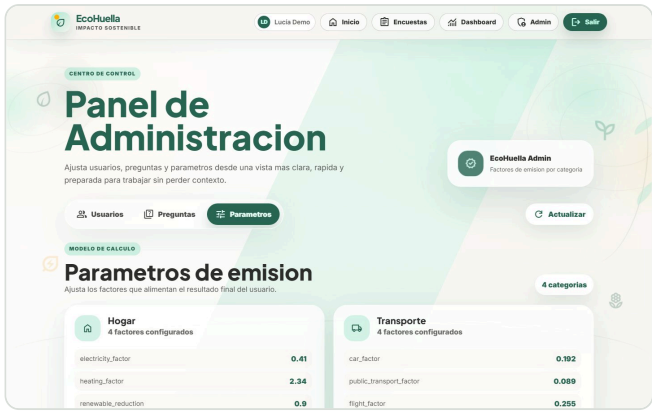


Figura 14. Pestaña Parámetros.

> **Atención: las pestañas Preguntas y Parámetros no alteran el cuestionario**
>
> Editan datos guardados en MongoDB, pero el cuestionario real lee `src/data/questions.json` y el cálculo usa constantes escritas en `quizResults.js`. Ver el detalle en la sección 14 (hallazgo C4).

6.5.9 Estructura común (Layout)

Todas las pantallas internas comparten la cabecera con el logo, el chip de usuario (abre el perfil), los accesos *Inicio*, *Encuestas*, *Dashboard*, *Admin* (solo administradores) y *Salir*; una capa decorativa de íconos de naturaleza al fondo; y un pie visible desde pantallas medianas.

6.6 Diseño visual y componentes de interfaz

La identidad visual se define en `tailwind.config.js` mediante *tokens* (nombres semánticos para colores, radios y tipografías), que luego se usan como clases (`bg-primary`, `text-on-surface`…). Las tipografías son **Plus Jakarta Sans** para títulos e **Inter** para texto.

primary	primary-container	tertiary-container	secondary	surface	on-surface
#296654	#acead3	#feb300	#1a4314	#f8f6f2	#2e2f2d

- **Componentes reutilizables:** `Button` (variantes primary, secondary y ghost; tamaños sm, md y lg) y `Card`. Son pocos: la mayor parte de los estilos de cada pantalla vive en clases propias de `index.css` (`.signup-card`, `.dashboard-hero`, `.admin-*`…).
- **Animaciones:** entradas suaves (`animate-rise`, `animate-pop`), flotación de íconos y elevación al pasar el cursor (`hover-lift`).
- **Responsivo:** la navegación se apila en pantallas pequeñas y las tarjetas pasan a una columna.



Figura 15. Vista móvil: selector.

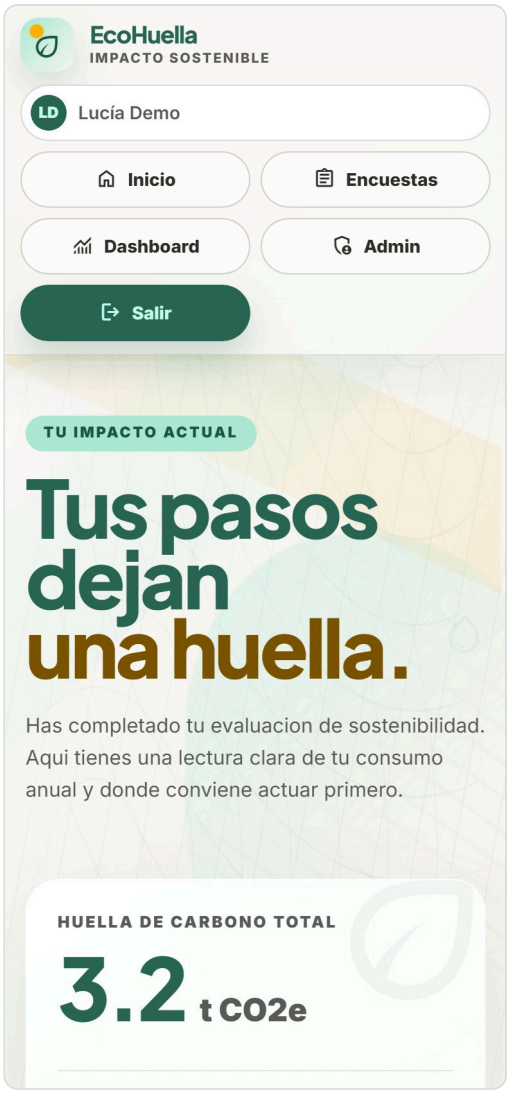


Figura 16. Vista móvil: dashboard.

6.7 Datos estáticos que alimentan el cuestionario

`src/data/questions.json` define los módulos, sus preguntas y, dentro de cada opción, los números que usa el cálculo. `surveyTracks.js` agrupa los módulos en recorridos:

Recorrido	Módulos	Preguntas	Preguntas incluidas
ambiental	hogar, transporte, alimentación, residuos	14	Personas, electricidad, calefacción, renovables · medio, km/semana, vuelos, ocupación · dieta, procedencia, desperdicio · reciclaje, compras, reparación
agua	agua	3	Veces por semana de lavarropas, riego y lavado de auto
desayuno	desayuno	4	Bebida, comida, origen y desperdicio

Cada opción de una pregunta tipo *select* puede llevar uno o varios de estos campos, según la categoría: `factor`, `value`, `impact_modifier`, `base_yearly_kg`, `extra_kg_co2`, `divisor`, `liters_per_km` y `breakfast_kg_co2_week`. Ver el significado de cada uno en la sección 8.

> **Respuestas guardadas por texto**
>
> Para las preguntas de opciones, lo que se almacena como respuesta es el **texto de la etiqueta** (por ejemplo "Gas Natural"), no un identificador. El cálculo vuelve a buscar la opción por ese texto. Cambiar una etiqueta en el JSON deja sin coincidencia a los resultados ya guardados.

El archivo `daily-tasks.json` (calculadora diaria con 5 momentos: desayuno, traslado, almuerzo, entrenamiento/hobby y cena) y los componentes `TaskSelector` y `DailyDashboard` que lo consumen **no están conectados a ninguna ruta**: son código de una funcionalidad no activada.

6.8 Aplicación instalable (PWA) y funcionamiento sin conexión

- **Manifiesto** (`manifest.webmanifest`): nombre, modo *standalone* (sin barra del navegador), orientación vertical, color de tema `#296654` e íconos. Permite "instalar" la aplicación en el teléfono o el escritorio.
- **Service worker** (`sw.js`, caché `ecohuella-v2`): precarga la estructura base; para navegaciones usa *red primero* con respaldo en la copia guardada de `index.html`; para archivos estáticos del mismo origen usa *caché primero*; las llamadas a `/api` nunca se guardan.

> **Límite a tener en cuenta**
>
> La aplicación no funciona realmente sin conexión: sin red no hay login, resultados ni encuestas. El service worker solo garantiza que la interfaz base abra. Además, las fuentes e íconos vienen de Google Fonts, que el service worker no guarda (solo almacena recursos del mismo origen); con una primera visita sin red, los íconos pueden verse como texto.

7. Flujo completo del usuario

El recorrido típico de una persona nueva atraviesa siete estados. El siguiente diagrama los resume y la tabla posterior indica qué ocurre en cada uno, tanto en el navegador como en el servidor.

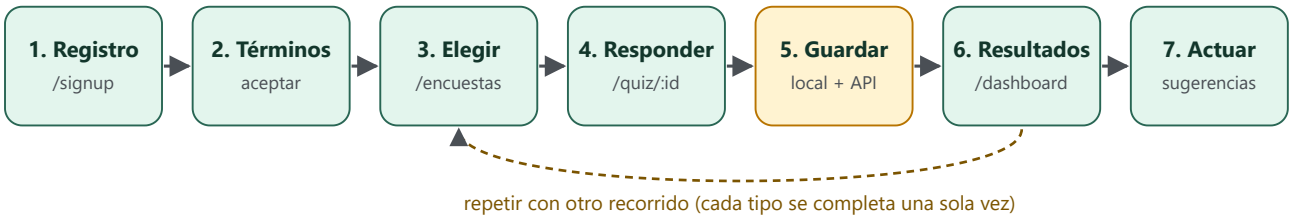


Figura 17. Secuencia de pantallas de una persona usuaria.

Paso	En el navegador	En el servidor
1. Registro	Valida el formulario, guarda token y usuario, marca términos pendientes y limpia datos locales previos.	`POST /auth/signup`: verifica que correo y usuario no existan, crea el usuario (contraseña hasheada) y devuelve un JWT.
2. Términos	Debe aceptar; se guarda el id en `localStorage`. Si rechaza, cierra sesión.	No interviene (la aceptación no se guarda en el servidor).

Paso	En el navegador	En el servidor
3. Elegir	Consulta resultados previos para deshabilitar recorridos ya completados.	`GET /results` devuelve los resultados del usuario (más recientes primero).
4. Responder	Recorre las preguntas del recorrido con barra de progreso.	`GET /results` otra vez, para impedir repetir una encuesta ya hecha.
5. Guardar	Guarda el intento en el historial local y envía respuestas y resultados.	`POST /results`: responde 409 si ya existe una encuesta de ese tipo; si no, crea el documento.
6. Resultados	Recalcula y muestra total, desglose, planetas e historial.	`GET /results`.
7. Actuar	Lee las dos recomendaciones de la categoría con mayor impacto.	—

7.1 Flujos secundarios

Recuperar la contraseña

1. En el login, "¿Olvidaste tu contraseña?" abre el modal.
2. Se ingresa el correo; el servidor genera un código de 6 dígitos y lo envía por e-mail.
3. La persona escribe el código y la nueva contraseña.
4. El servidor valida el código (hash + vencimiento) y actualiza la contraseña.

Administrar la plataforma

1. Un usuario con rol *admin* ve el acceso "Admin" en la cabecera.
2. En `/admin` gestiona usuarios (editar, cambiar rol, eliminar), preguntas y parámetros.
3. Cada acción llama a un endpoint protegido por `auth` + `isAdmin`.

8. El motor de cálculo

El cálculo vive íntegramente en el navegador, en `src/utils/quizResults.js` (función `calculateQuizResults`). Recibe las respuestas, busca en `questions.json` la opción elegida y aplica fórmulas simples de la forma *actividad* × *factor de emisión*. El servidor no recalcula: guarda lo que envía el cliente.

8.1 Fórmulas por categoría

Todos los resultados parciales se expresan en **kg de CO_2e por año** y recién al final se convierten a toneladas.

Hogar

```
hogar = ( kWh/mes × 12 × factor_electricidad + 50 × 12 × factor_calefacción ) ÷ personas × modificador_renovable
```

Pregunta	Opciones y valores
Electricidad	Bajo = 150 kWh/mes · Medio = 300 · Alto = 600 (factor 0,35 kg CO_2/kWh en los tres)
Calefacción	Gas natural = 2,02 · Eléctrica = 0,35 · Leña = 1,5 (se asume un consumo fijo de 50 unidades por mes)
Renovables	No uso = 1,00 · Termotanque solar = 0,85 · Paneles/energía verde = 0,70
Personas	Divide el consumo del hogar entre quienes lo comparten (mínimo 1)

Transporte

```
transporte = km/semana × 52 × factor_medio × litros_por_km ÷ divisor_ocupación + km_vuelo × factor_vuelo
```

Pregunta	Opciones y valores
Medio principal	Auto nafta = 2,31 kg CO_2/litro × 0,08 litros/km (≈ 0,185 kg/km) · Auto diésel = 2,68 × 0,07 (≈ 0,188 kg/km) · Transporte público = 0,05 kg/km · Bicicleta/a pie = 0
Kilómetros por semana	Deslizador de 0 a 500 (por defecto 50 si no se responde)
Ocupación	Viaja solo = ÷1 · 1 acompañante = ÷2 · 3 o más = ÷3,5
Vuelos del último año	Ninguno = 0 · Cabotaje = 2.000 km × 0,15 · Internacionales = 15.000 km × 0,18

Alimentación

```
alimentación = ( base_dieta + extra_desperdicio_anual ) × modificador_procedencia + desayuno
```

Pregunta	Opciones y valores
Dieta (kg/año)	Carnívora diaria = 2.500 · Carnívora moderada = 1.600 · Vegetariana = 1.000 · Vegana = 700
Procedencia	Siempre local = 0,90 · A veces = 1,00 · Supermercado/importado = 1,15

Pregunta	Opciones y valores
Desperdicio	Casi nada = 0 · Un plato = 50 · Bastante = 150 (kg CO_2e por año)

Residuos y consumo

```
residuos = ( 300 + extra_compras ) × modificador_reciclaje × modificador_reparación
```

Pregunta	Opciones y valores
Compras de ropa/tecnología	Solo si es necesario = +100 · Frecuencia normal = +300 · Muy frecuente = +800
Reciclaje	Siempre = 0,80 · A veces = 0,95 · Nunca = 1,00
Reparación / segunda mano	Casi siempre = 0,90 · A veces = 0,95 · Prefiero nuevo = 1,00

Desayuno y agua (recorridos complementarios)

```
desayuno = ( kg_bebida_semana + kg_comida_semana ) × 52 × modificador_origen +
extra_desperdicio_anual
agua (litros/año) = ( veces_lavarropas × 70 + veces_riego × 100 + veces_lavado_auto × 200 ) × 52
```

El resultado del desayuno se **suma a la categoría alimentación**. Los 70, 100 y 200 litros por uso del recorrido de agua son constantes del código; el agua no suma a la huella de carbono, se informa aparte.

8.2 Métricas derivadas

Métrica	Cálculo	Observación
Total (t CO_2e/año)	(hogar + transporte + alimentación + residuos) ÷ 1000, a 1 decimal	El redondeo a un decimal se aplica antes de las métricas siguientes.
Diferencia vs. media	(total − 4,7) ÷ 4,7 × 100	Negativa = por debajo de la media.
Planetas Tierra	total ÷ 1,5	1,5 t se toma como umbral sostenible por persona.
Categoría crítica	La de mayor valor entre las cuatro	Determina qué recomendaciones se muestran.

La comparación con la media y los planetas solo se muestran para la **encuesta ambiental**: los recorridos de agua y desayuno miden una parte de la huella y no son comparables con el total anual.

8.3 Ejemplo resuelto paso a paso

Se ejecutó la función real del proyecto (ya corregida) con el siguiente perfil: 3 personas en el hogar, consumo medio de electricidad, calefacción a gas natural, sin renovables, auto a nafta, 80 km por semana, viaja con 1 acompañante, un vuelo de cabotaje, dieta carnívora moderada, compra local "a veces", desperdicio "un plato o restos pequeños", recicla "a veces", compras "frecuencia normal" y repara "a veces".

Categoría	Operación	Resultado
Hogar	(300 × 12 × 0,35 + 50 × 12 × 2,02) ÷ 3 × 1,0 = (1.260 + 1.212) ÷ 3	824 kg

Categoría	Operación	Resultado
Transporte	80 × 52 × (2,31 × 0,08) ÷ 2 + 2.000 × 0,15 = 384 + 300	684 kg
Alimentación	(1.600 + 50) × 1,0	1.650 kg
Residuos	(300 + 300) × 0,95 × 0,95	542 kg
Total	3.700 kg ÷ 1000	**3,7 t CO_2e**
Diferencia vs. media	(3,7 − 4,7) ÷ 4,7 × 100	−21 %
Planetas	3,7 ÷ 1,5	2,5

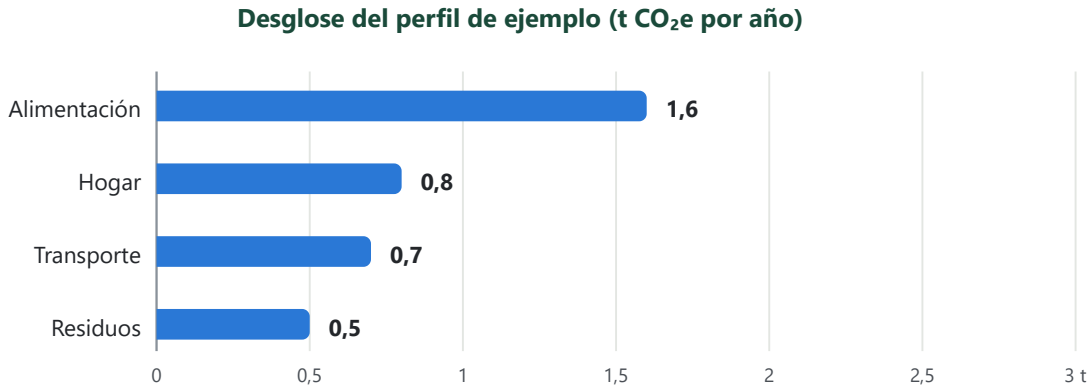


Figura 18. Perfil de ejemplo (3,7 t en total): la alimentación aporta más del 40 %; el transporte, con auto y un vuelo de cabotaje, 0,7 t.

8.4 Resultados con tres perfiles distintos

Para ver el rango que produce el modelo corregido se ejecutó la misma función con tres perfiles construidos con las opciones extremas del cuestionario.

Perfil	Hogar	Transp.	Aliment.	Residuos	Total (t)	vs. media	Planetas
Bajo impacto bicicleta, vegetariano, paneles, recicla siempre	0,1	0,0	0,9	0,3	**1,3**	−72 %	0,9
Perfil de ejemplo auto nafta 80 km/sem, dieta moderada	0,8	0,7	1,6	0,5	**3,7**	−21 %	2,5
Alto impacto diésel 300 km/sem, vuelo internacional, carne diaria	3,7	5,6	3,0	1,1	**13,5**	+187 %	9,0

> **Lectura de estos números**
>
> El perfil de ejemplo —una persona común con auto— resulta con 3,7 t, algo por debajo de la media de referencia de 4,7 t; el de alto impacto (diésel, 300 km por semana, vuelo internacional y dieta con carne diaria) llega a 13,5 t. Antes de la corrección del transporte y del desperdicio (hallazgos C1 y C2, sección 14) estos mismos perfiles daban 10,7 t y 61,2 t.

8.5 Qué se guarda en el servidor

Al terminar una encuesta, `buildQuizResultPayload` arma el documento que se envía a `POST /results`:

Campo	Contenido
`answers`	Respuestas tal cual (texto de opción o número) más `_surveyType`.
`results`	`housing_kg`, `transport_kg`, `food_kg`, `waste_kg`, `total_kg`, `total_tonnes`, `planets_needed`, `percentage_vs_average`.
`metadata`	`survey_type`, `household_size`, `renewable_energy` (booleano), `diet_type`, `transport_main`, `completed_at`.

9. El backend en detalle

9.1 Servidor Express (`server.js`)

El servidor se configura en este orden:

1. Carga variables de entorno con `dotenv`, comprueba que exista la clave de firma JWT (en producción el servidor no inicia sin `JWT_SECRET`) y crea la aplicación Express.
2. Habilita CORS (restringido a los orígenes de `CORS_ORIGIN` si está definido) y `express.json({ limit: "14mb" })` (el límite amplio permite la foto de perfil en base64).
3. Publica rutas de salud: `/`, `/api/health` (informa si la base está conectada) y `/healthz` / `/saludz` (usadas por Render).
4. Se conecta a MongoDB con `await connectDB()`. Si falla, **el servidor igualmente arranca**.
5. Aplica `requireDatabase` a todo `/api`: mientras no haya conexión responde **503** con un mensaje claro.
6. Monta los cinco grupos de rutas, un manejador 404 y un manejador global de errores (500 genérico).

9.2 Modelos de datos

User (colección `users`)

Campo	Tipo	Reglas
`name`	String	Obligatorio.
`username`	String	Obligatorio, único, minúsculas, 3–24 caracteres, solo letras/números/guion bajo.
`email`	String	Obligatorio, único, minúsculas, validado por expresión regular.
`password`	String	Obligatorio, mínimo 6, `select:false` (no se devuelve por defecto); se hashea antes de guardar.
`age`, `birthDate`	Number, Date	Edad 0–120.
`sex`	String	female, male, prefer_not_say o vacío.
`city`, `province`, `country`	String	País por defecto "Argentina".

Campo	Tipo	Reglas
`uccuyoLevel`	String	jardin, primario, secundaria, universidad o vacío.
`occupation`, `sustainabilityGoal`	String	Texto libre / clave de meta.
`householdSize`	Number	1–20, por defecto 1.
`avatarUrl`, `avatarZoom`, `avatarOffsetX/Y`	String, Number	Foto en base64; zoom 1–3; desplazamientos ±50.
`resetPasswordToken`, `resetPasswordExpires`, `resetPasswordAttempts`	String, Date, Number	`select:false`: hash SHA-256 del código, su vencimiento y los intentos fallidos (máximo 5).
`role`	String	user (por defecto) o admin.
`createdAt`, `updatedAt`	Date	Automáticos (`timestamps`).

QuizResult (colección `quizresults`)

Campo	Tipo	Descripción
`userId`	ObjectId → User	Dueño del resultado (obligatorio).
`answers`	Map de Mixed	Respuestas del cuestionario, incluido `_surveyType`.
`results`	Objeto	Valores numéricos calculados (kg, toneladas, planetas, % vs. media).
`metadata`	Objeto	Tipo de encuesta y datos de contexto.
`completedAt`	Date	Momento de finalización.

Question (colección `questions`) y EmissionParameter (`emissionparameters`)

Question guarda `id` (único), `module` (housing, transport, food, waste), `text`, `description`, `type` (select, number, range), `options` (etiqueta, valor, factor), `min` / `max` / `step` / `unit`, `parameters`, `order` y `active`.

EmissionParameter guarda una `category` única (housing, transport, food, waste) y un objeto `parameters` con cuatro grupos de factores numéricos (por ejemplo `electricity_factor` 0,41; `car_factor` 0,192; `omnivore_base` 2700), más `region`, `version` y `active`.

9.3 Catálogo de endpoints

Prefijo común: `/api`. "Token" significa que se requiere un JWT válido; "Admin" que además el rol debe ser *admin*.

Método	Ruta	Acceso	Función
POST	`/auth/signup`	Pública	Registra un usuario y devuelve token.
POST	`/auth/login`	Pública	Inicia sesión con correo o usuario.
POST	`/auth/forgot-password`	Pública	Genera y envía el código (responde igual exista o no la cuenta).
POST	`/auth/reset-password`	Pública	Cambia la contraseña con el código (máximo 5 intentos).
GET	`/auth/profile`	Token	Devuelve el perfil propio.
PUT	`/auth/profile`	Token	Actualiza el perfil propio.

Método	Ruta	Acceso	Función
GET	`/questions` · `/questions/:id`	Pública	Lista (filtros `module`, `active`) u obtiene una pregunta.
POST	`/questions`	Admin	Crea una pregunta.
PUT	`/questions/:id`	Admin	Actualiza una pregunta.
DELETE	`/questions/:id`	Admin	Elimina una pregunta.
GET	`/parameters` · `/parameters/:category`	Pública	Lista los parámetros activos o uno por categoría.
POST	`/parameters`	Admin	Crea parámetros.
PUT	`/parameters/:category`	Admin	Actualiza parámetros de una categoría.
GET	`/results`	Token	Resultados propios, más recientes primero.
GET	`/results/:id`	Dueño/Admin	Un resultado por id.
POST	`/results`	Token	Guarda un resultado (409 si el tipo ya existe).
PUT	`/results/:id`	Dueño/Admin	Actualiza un resultado (solo `answers`, `results` y `metadata`).
DELETE	`/results/:id`	Dueño/Admin	Elimina un resultado.
GET	`/results/admin/all`	Admin	Todos los resultados con nombre y correo del usuario.
GET	`/admin/users` · `/admin/users/:id`	Admin	Lista u obtiene usuarios (sin contraseña).
PUT	`/admin/users/:id`	Admin	Actualiza datos y rol de un usuario.
DELETE	`/admin/users/:id`	Admin	Elimina un usuario (no la propia cuenta).
PATCH	`/admin/users/:id/role`	Admin	Cambia el rol (un admin no puede quitarse el suyo).
GET	`/`, `/api/health`, `/healthz`, `/saludz`	Pública	Salud del servicio (sin prefijo `/api`, salvo `/api/health`).

9.4 Autenticación con JWT

EcoHuella usa **autenticación sin estado**: el servidor no guarda sesiones. Al iniciar sesión emite un *JSON Web Token* firmado que contiene `userId` y `role` y vence a los 7 días (configurable con `JWT_EXPIRE`). El cliente lo presenta en cada petición y el servidor solo debe verificar la firma.

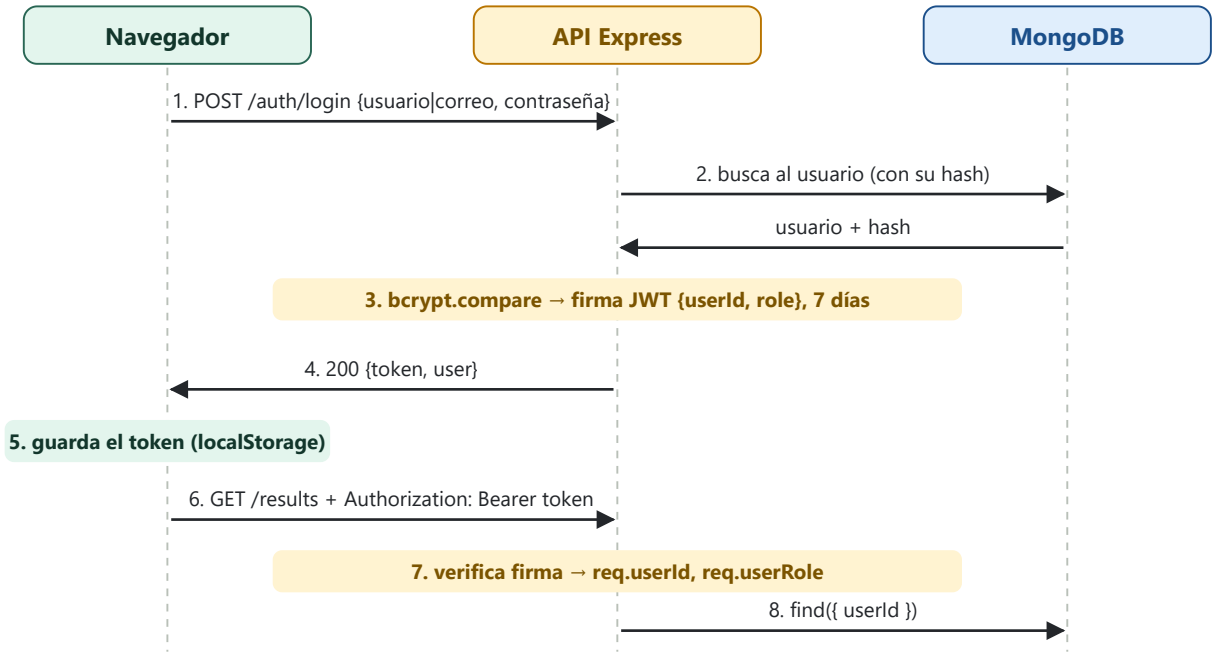


Figura 19. Inicio de sesión y uso posterior del token.

9.5 Recuperación de contraseña

1. **Solicitud** (`forgot-password`): busca la cuenta por correo; si no existe responde igual (200) para no revelar qué correos están registrados; si existe, genera un código aleatorio de 6 dígitos con `crypto.randomInt` , guarda *solo su hash SHA-256*, un vencimiento a 15 minutos y el contador de intentos en cero, y envía el correo con un límite de espera de 15 segundos.
2. **Restablecimiento** (`reset-password`): busca al usuario por correo y verifica que el código siga vigente; si el código es incorrecto lo rechaza y cuenta el intento (al 5.º fallo el código se invalida y debe pedirse otro). Con el código correcto exige una contraseña de 6 o más caracteres, guarda la nueva (se hashea con bcrypt) y borra el código.

Envío del correo (`utils/emailService.js`), por orden de prioridad: (1) API de **Resend** si existe `RESEND_API_KEY` ; (2) **Gmail** por SMTP con contraseña de aplicación (STARTTLS, puerto 587, resolución IPv4 forzada); (3) cualquier **SMTP** genérico. Si ninguno está configurado, la operación falla con un mensaje explícito.

9.6 Roles y permisos

Recurso	Sin sesión	Usuario	Admin
Registro, login, recuperación	✓	✓	✓
Ver / editar perfil propio	—	✓	✓
Leer preguntas y parámetros	✓	✓	✓
Crear / editar / borrar preguntas y parámetros	—	—	✓
Guardar y ver resultados propios	—	✓	✓

Recurso	Sin sesión	Usuario	Admin
Ver / modificar resultados ajenos	—	—	✓
Gestionar usuarios y roles	—	—	✓

Dos middlewares implementan esto en `middleware/auth.js`: `auth` (verifica el token y deja `userId` y `role` en la petición) e `isAdmin` (responde 403 si el rol no es admin). En los resultados, además, el controlador compara el dueño con `req.userId`. Un administrador tampoco puede eliminarse ni quitarse el rol a sí mismo.

9.7 Variables de entorno

Variable	Ámbito	Función
`MONGODB_URI` (o `MONGO_URI`, `DATABASE_URL`)	Backend	Cadena de conexión a MongoDB.
`JWT_SECRET`, `JWT_EXPIRE`	Backend	Clave de firma del token (obligatoria en producción) y su duración (por defecto 7d).
`PORT`, `NODE_ENV`	Backend	Puerto (5000 por defecto) y entorno.
`CORS_ORIGIN`	Backend	Orígenes permitidos por CORS, separados por coma. Vacío = cualquiera.
`SEED_ADMIN_PASSWORD`, `SEED_USER_PASSWORD`	Backend	Contraseñas de las cuentas de prueba del seed (opcional).
`RESEND_API_KEY`, `RESEND_FROM`	Backend	Envío de correo mediante Resend.
`GMAIL_USER`, `GMAIL_APP_PASSWORD`, `MAIL_FROM`	Backend	Envío por Gmail (SMTP).
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`	Backend	SMTP genérico alternativo.
`VITE_API_BASE_URL`	Frontend	URL base de la API en el build del frontend.

9.8 Script de datos iniciales (`seed.js`)

Ejecutado con `npm run seed`, **borra por completo** usuarios, preguntas y parámetros (`deleteMany({})`) y vuelve a crear: un usuario administrador y un usuario de prueba (sus credenciales figuran en `SETUP.md`), cinco preguntas de ejemplo y un documento de parámetros de emisión. Es útil para una instalación nueva. Para evitar accidentes, **se niega a ejecutarse si la base ya tiene usuarios** salvo que se indique `--force` (`npm run seed -- --force`).

10. Base de datos

Se utiliza **MongoDB**, una base de datos documental: en lugar de tablas y filas, guarda documentos JSON agrupados en colecciones. Mongoose aporta los esquemas y las validaciones.

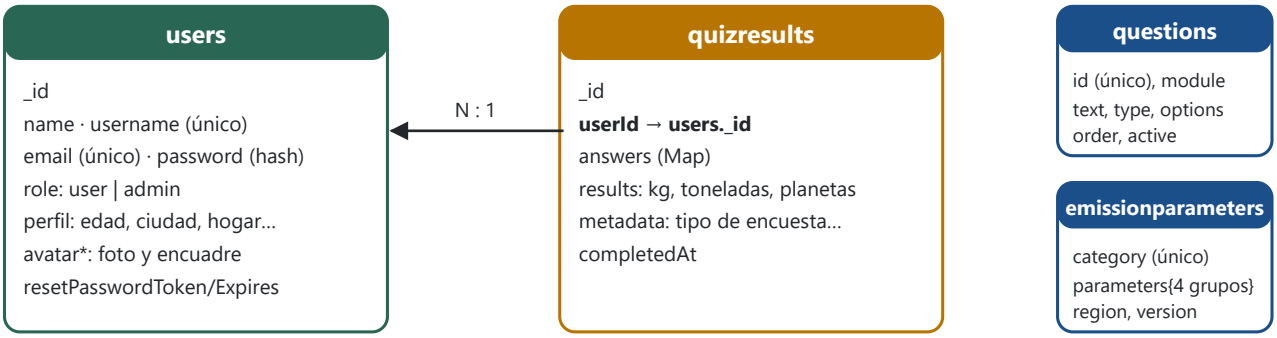


Figura 20. Las colecciones *questions* y *emissionparameters* no se relacionan con las demás ni son leídas por el cuestionario.

10.1 Reglas de integridad

- **Unicidad:** `username` y `email` (en usuarios), `id` (en preguntas) y `category` (en parámetros) tienen índice único.
- **Una encuesta por tipo y usuario:** se controla en el controlador antes de insertar (consulta por `userId` y tipo). No hay un índice único que lo garantice en la base, por lo que dos envíos simultáneos podrían duplicarla.
- **Sin borrado en cascada:** eliminar un usuario no elimina sus resultados.

10.2 Ejemplo de documento de resultado

```
{
  "_id": "…",
  "userId": "…",
  "answers": { "h_personas": "3", "h_calefaccion": "Gas Natural", "t_distancia_semanal": "80", "_surveyType":
"ambiental", … },
  "results": { "housing_kg": 824, "transport_kg": 5105, "food_kg": 4200, "waste_kg": 542,
               "total_kg": 10670, "total_tonnes": 10.7, "planets_needed": 7.1, "percentage_vs_average": 128 },
  "metadata": { "survey_type": "ambiental", "household_size": 3, "renewable_energy": false,
                "diet_type": "Carnívora moderada (3 veces por semana)", "transport_main": "Auto (Nafta)" },
  "completedAt": "2026-05-14T…Z"
}
```

11. Seguridad y privacidad

11.1 Medidas implementadas

Área	Qué se hace	Dónde
Contraseñas	Se guardan con hash bcrypt y sal (10 rondas); el campo no se devuelve en las consultas por defecto.	`models/User.js`
Sesión	JWT firmado con vencimiento; el rol viaja dentro del token y se verifica en cada petición protegida.	`middleware/auth.js`

Área	Qué se hace	Dónde
Autorización	Rutas de administración exigen rol admin en el servidor; los resultados solo son accesibles por su dueño o un admin.	`routes/*`, `quizResultController.js`
Recuperación	El código de 6 dígitos se guarda hasheado (SHA-256), vence a los 15 minutos, admite 5 intentos fallidos y se elimina al usarse. La respuesta no revela si la cuenta existe.	`authController.js`
Asignación de rol	El registro ignora cualquier campo `role` enviado por el cliente; todos los usuarios nuevos son *user*.	`authController.signup`
Secretos	`.env` está en `.gitignore`; hay archivos `.env.example` sin valores reales.	`.gitignore`
Disponibilidad	Si la base de datos cae, la API responde 503 con un mensaje claro en lugar de fallar de forma opaca.	`server.js`
Errores	El manejador global no expone la traza al cliente.	`server.js`
Clave de firma	Una única fuente para firmar y verificar tokens; en producción el servidor no inicia sin `JWT_SECRET`.	`config/jwt.js`
Administración	Un admin no puede eliminarse ni quitarse su propio rol; los resultados solo admiten modificar `answers`, `results` y `metadata`.	`userController.js`, `quizResultController.js`
Datos de prueba	`seed.js` se niega a borrar una base con usuarios salvo `--force`.	`seed.js`

11.2 Datos personales que trata la aplicación

Dado que el proyecto se enmarca en un curso de ética y trata datos de personas —incluidas, según el campo "nivel UCCuyo", potenciales personas menores de edad—, conviene explicitar qué información recolecta:

Categoría	Datos	Obligatorio
Identificación y contacto	Nombre completo, nombre de usuario, correo electrónico	Sí
Demográficos	Fecha de nacimiento, edad, sexo	Sí (en el registro)
Ubicación	Ciudad, provincia, país	Ciudad y provincia
Contexto	Nivel UCCuyo, ocupación, personas en el hogar, meta de sostenibilidad	Nivel UCCuyo
Imagen	Foto de perfil (guardada como imagen codificada en la base)	No
Comportamiento	Respuestas de las encuestas y resultados calculados	Al responder

> **Consentimiento sin registro en el servidor**
> La aceptación de los Términos y Condiciones se guarda únicamente en el `localStorage` del navegador. El servidor no conserva quién aceptó, cuándo ni qué versión. Esto implica que (a) no hay constancia verificable del consentimiento y (b) la persona debe volver a aceptar en cada navegador o dispositivo. Para un tratamiento de datos personales, lo recomendable es registrar la aceptación (fecha y versión) en el usuario.

12. Despliegue y operación

12.1 Cómo se publica

La guía `DEPLOY_GRATIS.md` describe un despliegue sin costo en tres servicios. En resumen:

1. **MongoDB Atlas:** crear un cluster gratuito, un usuario de base de datos y habilitar el acceso por red; copiar la cadena `mongodb+srv://…` apuntando a la base `eco-huella`.
2. **Render (backend):** nuevo *Web Service* con directorio raíz `backend`, build `npm install` y start `npm start`; variables `MONGODB_URI`, `JWT_SECRET` (obligatoria), `JWT_EXPIRE`, `NODE_ENV` y, recomendada, `CORS_ORIGIN` con la URL del frontend. Verificar con `/api/health`.
3. **Datos iniciales:** desde la consola de Render ejecutar `npm run seed` (solo en una base vacía; si ya tiene usuarios, el script se niega salvo que se agregue `-- --force`, que la borra).
4. **Vercel (frontend):** importar el repositorio, preset Vite, salida `dist`, variable `VITE_API_BASE_URL` apuntando a la API de Render.

El archivo `vercel.json` contiene un *rewrite* que devuelve `index.html` para cualquier ruta; sin él, recargar `/dashboard` daría un error 404 en Vercel.

12.2 Scripts disponibles

Comando	Efecto
`npm run dev`	Solo el frontend (Vite, puerto 8080).
`npm run dev:all`	Frontend y backend juntos mediante `start-dev.ps1` (verifica puertos y dependencias; PowerShell/Windows).
`npm run build` / `preview`	Compila para producción / sirve la compilación localmente.
`npm run lint`	Análisis estático con ESLint.
`cd backend && npm run dev`	API con recarga automática (`node --watch`).
`cd backend && npm run seed`	Carga datos iniciales; con usuarios existentes exige `-- --force` (destructivo).

12.3 Consideraciones operativas

- **Arranque en frío:** en el plan gratuito de Render el servicio se duerme tras un período sin tráfico; la primera petición puede tardar. La aplicación no muestra un estado de "despertando" al usuario.
- **Salud del servicio:** `/api/health` informa si la base está conectada; `/healthz` y `/saludz` sirven para monitoreo.

- **Compartir en desarrollo:** `vite.config.js` admite un host de túnel de Cloudflare y el script `start-dev.ps1` explica cómo publicar solo el puerto 8080.
- **Archivos de registro:** hay varios `*.log` en la raíz del proyecto; están excluidos por `.gitignore` y no deberían versionarse.

13. Calidad del código y estado del proyecto

13.1 Resumen

Aspecto	Estado	Detalle
Organización	Buena	Carpetas por responsabilidad; patrón ruta–controlador–modelo; hooks y contexto bien delimitados.
Análisis estático	Configurado	ESLint con reglas de React Hooks; un commit lo dedica a estabilizarlo.
Pruebas automatizadas	Ausentes	No existen archivos de prueba en el repositorio. Las comprobaciones de esta revisión (28 del backend y 6 de navegación) se ejecutaron con guiones externos; conviene incorporarlas, empezando por el motor de cálculo.
Documentación	Buena	`README.md`, `SETUP.md` y `DEPLOY_GRATIS.md` describen el estado real (actualizados en esta revisión).
Código sin uso	Presente	`TaskSelector`, `DailyDashboard`, `daily-tasks.json`, `App.css`, `Header.jsx`, `IconWrapper.jsx` y `ProgressBar.jsx` (vacíos), `express-validator`, `getAllResults`.
Archivos mezclados	A depurar	Registros `.log` (ignorados por git) y, sin versionar en la raíz, un `.docx` y la carpeta `tools/`.

13.2 Cómo evolucionó (historial de commits)

Período	Trabajo realizado
26 abr – 6 may	Configuración inicial (ESLint, `.gitignore`), base del frontend y guardado de la configuración de ejecución local.
8 may	Preparación del despliegue gratuito (Vercel/Render), rutas de salud, tolerancia a caída de MongoDB, recuperación de contraseña con código por correo (Resend y Gmail), conversión a PWA y mejoras del panel admin.
11 may	Registro y preguntas de agua, historial de encuestas por usuario, selector de recorridos y regla de una sola vez por encuesta.
13 – 14 may	Estabilización de lint y del cuestionario, corrección de navegación y de resultados, nivel UCCuyo editable por admin, fondos visuales y mensaje de credenciales inválidas.
14 oct	Revisión técnica: corrección del cálculo, de la navegación al recargar, de los mensajes y de varios puntos de seguridad (sección 14). Cambios sin commitear.

Total: 48 commits en 19 días. El estado actual incluye además cambios sin commitear: los dos archivos de navegación que ya estaban modificados (`ProtectedRoute.jsx` y `TermsAndConditionsPage.jsx`) y las correcciones de la revisión (sección 14).

14. Correcciones aplicadas y observaciones pendientes

La primera revisión del código (14/10/2026) detectó errores y riesgos. En una segunda etapa se corrigieron los que son **defectos claros y no requieren decisiones de producto**, y se verificó el resultado. Esta sección documenta qué se cambió, cómo se comprobó, cuánto cambian los resultados y qué queda pendiente. El resto del informe describe la aplicación **ya con estas correcciones**.

Alta afectaba la validez del producto o la seguridad de los datos · **Media** degradaba la experiencia o ampliaba riesgos · **Baja** mejora de calidad.

14.1 Correcciones aplicadas

> **C1 y C2 · El modelo de cálculo estaba mal calibrado**
>
> **C1:** los factores del auto (2,31 y 2,68) son kg de CO_2 por *litro* de combustible, pero se multiplicaban por *kilómetros*: unas 12 veces el `car_factor` de 0,192 kg/km que define el propio backend. **C2:** el valor de desperdicio ("Un plato" = 50, "Bastante" = 150) se multiplicaba por 52, aunque su magnitud corresponde a un valor *anual* (igual que en el recorrido de desayuno). Ambos están corregidos.

ID	Gravedad	Problema	Qué se cambió	Archivos
C1	Alta	Transporte en auto sobrestimado ~12×.	Las opciones de auto incorporan `liters_per_km` (0,08 nafta; 0,07 diésel). El cálculo usa *factor* × *consumo* ≈ 0,185 y 0,188 kg/km, coherente con el backend.	`questions.json`, `quizResults.js`
C2	Alta	Desperdicio de comida multiplicado por 52.	Se trata como valor anual, sin multiplicar.	`quizResults.js`
B2	Alta	Recargar una pantalla protegida terminaba siempre en `/dashboard`.	Nuevo estado `checkingAuth`: las rutas protegidas y la pantalla de términos esperan a que se confirme la sesión antes de decidir a dónde ir.	`AuthContext.jsx`, `ProtectedRoute.jsx`, `TermsAndConditionsPage.jsx`
S1	Alta	Código de recuperación sin límite de intentos.	Máximo 5 intentos fallidos; luego el código se invalida y debe pedirse otro (campo `resetPasswordAttempts`).	`authController.js`, `User.js`
C3	Media	Recorridos parciales comparados con la huella anual completa ("98 % por debajo de la media").	Para agua y desayuno no se muestran la comparación ni los planetas: se explica que el resultado es parcial. Sin emisiones no se muestran recomendaciones.	`ResultsDashboard.jsx`
B1	Media	Mensajes del servidor con codificación corrupta.	Se repararon 8 mensajes (10 palabras) y el archivo quedó en UTF-8 correcto.	`authController.js`

ID	Gravedad	Problema	Qué se cambió	Archivos
S2	Media	La recuperación revelaba qué correos están registrados.	`forgot-password` responde lo mismo exista o no la cuenta. (El registro sigue avisando que un correo ya está en uso.)	`authController.js`
S4	Media	Clave JWT inconsistente entre firmar y verificar.	Una sola fuente (`config/jwt.js`): clave de desarrollo solo fuera de producción; en producción el servidor no inicia sin `JWT_SECRET`.	`jwt.js`, `auth.js`, `authController.js`, `server.js`
S5	Media	`seed.js` borraba todo sin avisar y publicaba contraseñas.	Se niega a ejecutarse si ya hay usuarios salvo `--force`; las contraseñas de prueba pueden venir de variables de entorno.	`seed.js`
S3	Media	CORS abierto a cualquier origen.	Ahora configurable con `CORS_ORIGIN` (uno o varios orígenes). Sin definirlo conserva el comportamiento anterior: hay que configurarlo en producción.	`server.js`
B5	Baja	El dashboard mostraba 0,0 m³ de agua al ver la encuesta ambiental.	Toma el dato de la encuesta de agua más reciente; si no existe, indica que debe completarse.	`ResultsDashboard.jsx`
B3	Baja	El login recortaba espacios de la contraseña y el registro no.	El login ya no modifica la contraseña.	`AuthContext.jsx`
S7	Baja	`PUT /results/:id` aceptaba cualquier campo (p. ej. `userId`).	Solo acepta `answers`, `results` y `metadata`.	`quizResultController.js`
S8	Baja	La validación de correo rechazaba dominios como `.info` o `.tech`.	La expresión regular admite dominios de nivel superior de 2 o más letras.	`User.js`
S9	Baja	Un admin podía eliminarse o quitarse el rol a sí mismo.	Ambas acciones responden 400.	`userController.js`
—	Baja	Aviso del editor en `Layout.jsx`: la clase `flex-grow` puede escribirse `grow` (Tailwind 4).	Se reemplazó por `grow`.	`Layout.jsx`
—	Baja	README y SETUP desactualizados o inexactos.	Describen el estado real (backend, panel admin, `seed --force`, `CORS_ORIGIN`).	`README.md`, `SETUP.md`

14.2 Cómo se verificó

Verificación	Resultado
Análisis estático (`npm run lint`)	Sin errores ni advertencias.
Compilación (`npm run build`)	Correcta.

Verificación	Resultado
Backend contra una base de datos temporal (28 comprobaciones)	28 de 28 correctas: mensajes con acentos, correo `.info`, token sin `JWT_SECRET` en desarrollo, respuesta genérica de recuperación, 5 intentos y bloqueo, reutilización de código, `userId` inmodificable, protecciones del admin, CORS por origen, negativa a iniciar en producción sin clave y protección del `seed`.
Navegación en navegador real (6 comprobaciones)	Recargar `/encuestas`, `/admin`, `/quiz/desayuno` y `/` conserva la ruta; sin sesión redirige a `/login`; sin aceptar términos, a `/terminos-y-condiciones`.
Cálculo	Se volvieron a ejecutar los tres perfiles y los recorridos parciales con la función corregida (tabla siguiente).

Las pruebas se ejecutaron con guiones externos al repositorio y no usaron los datos reales: la base temporal se eliminó al terminar y el envío de correos estaba deshabilitado.

14.3 Efecto de las correcciones en los resultados

Perfil	Transporte antes → ahora	Aliment. antes → ahora	Total antes → ahora	vs. media	Planetas
Bajo impacto	0,0 → 0,0	0,9 → 0,9	1,3 → **1,3 t**	−72 % → −72 %	0,9 → 0,9
Perfil de ejemplo	5,1 → 0,7	4,2 → 1,6	10,7 → **3,7 t**	+128 % → −21 %	7,1 → 2,5
Alto impacto	44,5 → 5,6	11,8 → 3,0	61,2 → **13,5 t**	+1.202 % → +187 %	40,8 → 9,0

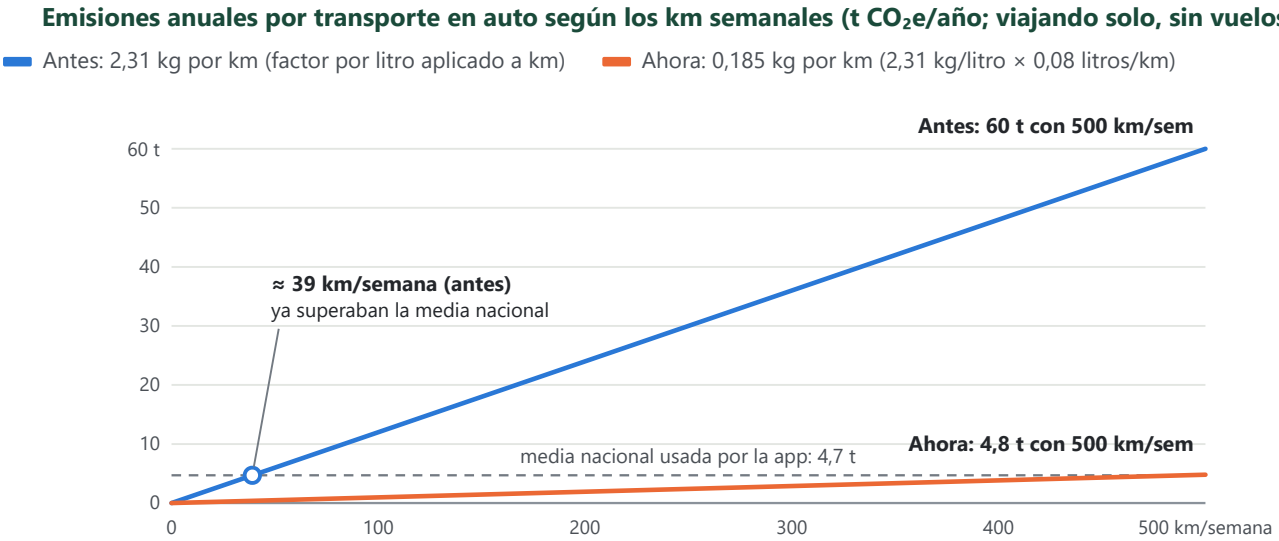


Figura 21. La corrección reduce unas 12 veces la pendiente del cálculo: con la misma distancia, la emisión pasa de la línea azul a la naranja.

Supuesto que debe validarse

El consumo de combustible (0,08 L/km en nafta y 0,07 L/km en diésel) es una estimación razonable para el parque automotor, pero es una **elección de modelado** y no un dato del proyecto original. Está escrita en `questions.json` y puede ajustarse sin tocar el código; conviene respaldarla con una fuente citada.

14.4 Observaciones pendientes

Quedan sin resolver los puntos que requieren una decisión de producto, una dependencia nueva o un cambio de diseño mayor.

ID	Gravedad	Hallazgo	Recomendación
C4	Media	**El panel de preguntas y parámetros está desconectado del cuestionario.** El quiz lee `questions.json` y constantes del código; las colecciones *questions* y *emissionparameters* solo las usa el panel admin. Los valores también divergen entre ambas fuentes (por ejemplo electricidad 0,41 vs. 0,35).	Elegir una única fuente de verdad: que el cuestionario consuma la API y el cálculo use los parámetros guardados, o retirar el panel.
S10	Media	**El consentimiento de términos se guarda solo en el navegador** (ver sección 11.2): sin constancia verificable y se vuelve a pedir en cada dispositivo.	Guardar `termsAcceptedAt` y la versión en el usuario.
S3b	Media	**Sin límite general de peticiones ni cabeceras de seguridad**, y CORS sigue abierto hasta definir `CORS_ORIGIN`.	Definir `CORS_ORIGIN` en producción y sumar `helmet` y `express-rate-limit`.
S6	Media	**Token en localStorage por 7 días**, sin renovación ni revocación.	Evaluar cookies `httpOnly` o una duración menor con renovación.
C6	Baja	**Referencias y supuestos sin fuente:** 4,7 t, 1,5 t, consumos de combustible, 50 unidades de calefacción por mes, 70/100/200 litros y los factores de emisión.	Documentar fuentes y año en el JSON o en la ayuda de la aplicación.
C5	Baja	**Las respuestas se guardan por texto de etiqueta**: cambiar un texto del JSON invalida resultados históricos.	Asignar un identificador estable a cada opción.
B4	Baja	**"Volver a evaluar" no permite repetir**: lleva al selector, donde todo recorrido completado está deshabilitado.	Definir la política y reflejarla (cambiar el texto o permitir rehacer).
B6	Baja	**Un guardado fallido en el servidor es silencioso.**	Avisar el error y ofrecer reintento.
S11	Baja	**Foto de perfil en base64 dentro de MongoDB** (hasta ~8 MB).	Reducir la imagen en el cliente o usar almacenamiento de archivos.
B7	Baja	**Dependencia de Google Fonts** sin copia local: sin red, los íconos se ven como texto.	Alojar las fuentes en el proyecto.
B8	Baja	**Elementos inertes y deuda menor:** checkbox "Mantenerme conectado" sin efecto; textos sin tildes; archivos vacíos y código sin uso (sección 13); `express-validator` instalado sin usar; sin pruebas automatizadas en el repositorio.	Limpiar, unificar la redacción y agregar pruebas al motor de cálculo y a la API.

14.5 Plan de acción sugerido

1 · Inmediato (horas)

- Definir `CORS_ORIGIN` y `JWT_SECRET` en Render.
- Validar y citar el consumo de combustible (C6).
- Agregar pruebas de los perfiles de referencia del cálculo.

2 · Corto plazo (días)

- S3b: `helmet` y limitador de peticiones.
- B6: avisar y reintentar cuando falla el guardado.
- B7: alojar fuentes localmente.

3 · Mediano plazo

- C4: una única fuente de verdad para preguntas y factores.
- S10: registrar el consentimiento en el servidor.
- C5: identificadores estables en las opciones.

4 · Evolución del producto

- Permitir repetir encuestas y comparar en el tiempo (B4).
- Exportar el informe personal (PDF/CSV).
- Mejorar el modo sin conexión.

15. Cómo ejecutarla en una computadora

15.1 Requisitos

- **Node.js 20 o superior** y npm 10 o superior.
- **MongoDB** en ejecución local, o una cadena de conexión de MongoDB Atlas.

15.2 Pasos

```
# 1. Instalar dependencias (frontend y backend)
npm install
cd backend && npm install && cd ..

# 2. Crear backend/.env con, como mínimo:
PORT=5000
MONGODB_URI=mongodb://localhost:27017/eco-huella
JWT_SECRET=una_clave_larga_y_secreta
JWT_EXPIRE=7d

# 3. (Solo base vacía) cargar datos iniciales
cd backend && npm run seed && cd ..

# 4. Levantar frontend y backend juntos (Windows/PowerShell)
npm run dev:all
# Abrir http://localhost:8080/login
```

15.3 Verificación y problemas frecuentes

Síntoma	Causa probable y solución
El login muestra "No se pudo conectar con el servidor"	El backend no está encendido. Probar `http://localhost:5000/api/health`.
La API responde 503 "La base de datos no esta conectada"	MongoDB no está activo o `MONGODB_URI` es incorrecta.
`npm run seed` dice que la base ya tiene usuarios	Es una protección. Si de verdad quieres reiniciarla, ejecuta `npm run seed -- --force` (borra usuarios, preguntas y parámetros).
Puerto 5000 u 8080 en uso	Cerrar el proceso anterior o cambiar `PORT` (y el destino del proxy en `vite.config.js`).
El servidor se cierra al iniciar en producción	Falta `JWT_SECRET` en las variables de entorno (es obligatorio con `NODE_ENV=production`).
No llega el código de recuperación	Configurar `RESEND_API_KEY` o `GMAIL_USER` + `GMAIL_APP_PASSWORD`.

16. Glosario

Término	Significado
API REST	Conjunto de direcciones (URL) y operaciones (GET, POST, PUT, DELETE) mediante las cuales el frontend pide o envía datos al servidor en formato JSON.
bcrypt	Algoritmo que transforma una contraseña en un código irreversible (hash) con una "sal" aleatoria, de modo que no se guarda la contraseña real.
CO_2e (CO_2 equivalente)	Unidad que expresa el efecto de distintos gases de efecto invernadero como si fueran dióxido de carbono.
CORS	Mecanismo del navegador que controla qué sitios pueden llamar a una API.
Factor de emisión	Cantidad de CO_2e que genera una unidad de actividad (por ejemplo, kg por kWh o por km).
Hash	Resultado de aplicar una función matemática unidireccional a un dato; permite compararlo sin conocerlo.
JWT	*JSON Web Token*: credencial firmada que identifica a un usuario y su rol sin que el servidor deba guardar sesiones.
localStorage	Almacenamiento del navegador, propio de cada sitio y dispositivo, que persiste entre visitas.
Middleware	Función que se ejecuta entre la petición y el controlador (por ejemplo, para verificar el token).
MongoDB / Mongoose	Base de datos documental y la biblioteca que define sus esquemas desde Node.js.
PWA	*Progressive Web App*: sitio web que puede instalarse y comportarse como una aplicación.
Rate limiting	Límite de peticiones permitidas en un lapso, para frenar abusos y ataques de fuerza bruta.
SPA	*Single Page Application*: aplicación que carga una sola vez y cambia de pantalla sin recargar la página.

Término	Significado
Service worker	Script que el navegador ejecuta en segundo plano; aquí gestiona la caché de la aplicación instalable.
Vite	Herramienta que sirve la aplicación en desarrollo y la empaqueta para producción.
XSS	Ataque que inyecta código en una página para ejecutarlo en el navegador de otra persona; puede robar datos guardados en el navegador, como un token.

Conclusión

EcoHuella es un MVP completo y bien organizado: cubre el ciclo de registro, encuesta, cálculo, resultados y administración con una arquitectura clara y un despliegue documentado. La revisión encontró defectos en la calibración del cálculo, en la navegación al recargar y en puntos de seguridad; los defectos claros ya están corregidos y verificados. Lo que sigue pendiente depende sobre todo de decisiones de producto: **conectar el panel de administración con el cuestionario** (C4), **registrar el consentimiento en el servidor** (S10), **documentar las fuentes del modelo** (C6) y completar el endurecimiento de seguridad (S3b y S6).